

GraphOne / FrontierAtlas Intelligence Graph

Technical Architecture — Data Ingestion Pipeline

1. Scale Strategy: Collecting 500,000+ Records

The trial implementation (this submission) collects ~1,000–2,000 records per vertical using the same architecture designed to scale to 500,000+ without code changes — only infrastructure. The key design decision enabling this is **source selection by API quality, not by convenience**: every vertical is built against a structured, high-volume, low-friction data source rather than scraping individual pages one at a time.

Per-vertical scale path:

- **Research Papers**: arXiv's API paginates in batches of 100–2000 across 150+ categories. Scaling from ~1,000 to 500,000 papers is purely a matter of iterating more categories and raising the per-category target — the pagination and dedup logic already handle arbitrary volume.
- **Startups**: the `yc-oss.github.io` mirror exposes the entire YC portfolio (6,200+ companies) as pre-tagged static JSON. At full scale, the same pattern extends to additional structured directories (Crunchbase Open Data exports, AngelList/Wellfound public APIs, government business registries) fetched the same way — static bulk JSON/CSV pulls rather than page-by-page HTML scraping.
- **Products**: derived from startup records via LLM extraction. At 500k scale, this becomes an embarrassingly parallel batch job: partition startups across N worker processes, each running an independent instance of the LLM orchestrator against its own shard.
- **News & Jobs**: scale via horizontal worker count, not per-worker complexity — each of the 5 sources gets its own async crawler instance; adding sources means adding workers, not rewriting logic.

Concurrency model:

All scrapers use Python `asyncio` + `aiohttp` with a bounded semaphore per data source (typically 5–8 concurrent connections) rather than one giant global pool. This is deliberate: different sources have wildly different rate limits (arXiv asks for 3s courtesy delay; GitHub allows 5,000 authenticated requests/hour; a news site's WAF may allow far less). At 500k scale, this same per-source semaphore pattern is distributed across multiple worker machines via a shared job queue (see Section 3), so total system throughput scales linearly with worker count while each individual source's rate limit is still respected globally, not per-worker.

2. Handling 413s and 429s Across Thousands of Extractions

429 (Rate Limited) — Exponential backoff with jitter, tier fallback

Every network call in the system (scrapers and LLM calls alike) routes through a shared `with_retries()` utility implementing exponential backoff ($\min(\text{max_delay}, \text{base_delay} * 2^{\text{attempt}}) + \text{random jitter}$), and honors a provider's `Retry-After` header when present rather than guessing. This was validated empirically during the trial: GitHub's search API (10 req/min unauthenticated) and Groq's free tier both triggered real 429s, and the same retry code recovered from both without manual intervention.

For the LLM orchestration layer specifically, retries are bounded per-tier (5 attempts) before falling through to the *next* provider in the chain (Gemini Flash → Groq → DeepSeek/OpenRouter) rather than retrying the same exhausted provider indefinitely. A single global pacing gate (`asyncio.Semaphore(1)` + minimum inter-call delay) serializes all LLM calls regardless of how many extraction tasks are logically "concurrent" — this was a direct fix after observing that even 5 concurrent workers could burst past a free-tier RPM limit faster than per-call retries could recover.

413 (Payload Too Large) — Pre-emptive chunking, not reactive handling

Rather than catching 413s after the fact, every LLM input is run through a `truncate_for_llm()` step before the request is ever sent, capping input at a configurable character budget (default 6,000 chars $\approx$ 1,500 tokens, well under any provider's smaller context windows once the prompt template is added). Truncation keeps both the head and tail of long text (70/30 split) rather than blindly cutting from the end, since articles and descriptions often carry distinct information at both ends. A 413 is still treated as a possible (non-retryable) failure path in code, in case a provider's real limit is stricter than assumed — but the design goal is that this path is never actually hit in practice.

3. Freshness Tracking Across Distributed Crawler Nodes

The trial implementation uses file-based JSONL checkpoints keyed by a natural unique ID per vertical (arXiv ID for papers, source URL for startups/products, article URL for news, job posting URL for jobs). On every run, already-seen IDs are loaded first and skipped, making each script independently idempotent and safely resumable — demonstrated in practice when the papers and products collection runs were interrupted by real network blips and resumed cleanly without duplicate work or lost progress.

At distributed, multi-node scale, this same idea moves from a local file to a shared, centralized deduplication store:

- **Dedup key store:** a Redis set (or a dedicated column with a unique index in the primary database) holding every processed record's natural key. Before a worker processes a URL/ID, it performs an atomic SADD (add-if-absent) — if the key already existed, the worker skips it. This is the same check-before-work pattern as the local JSONL approach, just backed by a store all workers share.
- **Job queue with visibility timeout:** a message queue (SQS-style) hands each crawl target to exactly one worker at a time; if a worker crashes mid-task, the message becomes visible again after a timeout rather than being silently lost or double-processed by two workers simultaneously.
- **24-hour freshness for News/Jobs specifically:** normalized `published_date` (parsed from explicit timestamps, relative phrases like "2 hours ago", or a last-seen heuristic when no date is present) is checked against a rolling 24h window at ingestion time, not at query time — stale records are dropped before they ever reach storage, keeping the freshness guarantee an ingestion invariant rather than a filter applied after the fact.

4. Storage Strategy

Layer	Choice	Justification
Primary operational store	PostgreSQL	Structured, schema-enforced records (matches the 4 canonical entity schemas exactly); native JSONB columns absorb source-specific variable fields (e.g. YC's tags, arXiv's categories) without a rigid migration for every new source; mature tooling for the dedup/unique-key constraints described in Section 3.
Graph relationships	Neo4j (or PostgreSQL + Apache AGE)	The core value of an "Intelligence Graph" is the edges, not just the nodes: Startup–Product ownership, Paper–GitHub repo links, Founder–Company relationships, and entity-resolution merge history are all naturally modeled as a graph. Neo4j's Cypher queries ("find all papers whose authors later founded a startup in the same category") are the kind of multi-hop query a relational JOIN chain handles poorly at scale.
Vector store	pgvector (co-located with Postgres) at moderate scale; Qdrant/Pinecone if embedding volume grows past single-node Postgres comfort	Semantic search over paper abstracts and startup descriptions (e.g. "find startups similar to this one") needs embedding similarity search, not keyword match. Starting co-located with Postgres (via pgvector) avoids a second system to operate until scale actually demands it — premature infrastructure complexity is a real cost on a small team.
Dedup/queue layer	Redis	Sub-millisecond set membership checks for the freshness/dedup pattern in Section 3, plus doubles as the job queue backing store; already the de facto standard for this exact access pattern.

Why not a single database for everything: a graph database optimized for relationship traversal is a poor fit for high-volume structured writes (millions of individual startup/paper inserts), and a relational database's JOIN performance degrades on the deep multi-hop queries a real "who influenced whom" graph question requires. Splitting

by access pattern — Postgres for structured writes and lookups, a graph layer for relationship queries, a vector layer for similarity search — is standard practice for systems combining these three query shapes, and each layer can scale independently as usage patterns reveal which one is the actual bottleneck.